



4.1. 0/1 Knapsack Problem

"Take it or leave it: Maximizing what matters."

Main Section:

 [4. Dynamic Programming](#)

Contents:

[1. Problem Definition](#)

[2. Workflow](#)

[Summary](#)

[References](#)

Earlier, we explored the classic Knapsack Problem using the Greedy approach. And honestly, Greedy works beautifully —*but only under one important condition*. You must be allowed to take fractions of objects. If an item is too large, Greedy simply slices part of it and places that fraction into the bag. Easy. Efficient. Elegant.

But now, let's remove that freedom. What if the objects are indivisible? *What if you must either take the entire object or leave it completely behind?*

No fractions. No cutting. No compromises. That changes everything. This version is called the **0/1 Knapsack Problem**. Why "0/1"? Because every object has only two possible states **0** → Not included, and **1** → Included

Simple idea. But suddenly, the Greedy strategy begins to fail. Why? Because Greedy only focuses on the *best immediate choice*. It looks at one object at a time and asks: *"What gives me the highest value right now?"*

But the 0/1 Knapsack Problem is not about immediate choices. It is about **combinations**. Sometimes, choosing a slightly worse object now, allows a much better combination later. And Greedy cannot see that future.

So instead of chasing local decisions, we need something smarter. Something that can systematically explore possibilities without becoming impossibly slow.

This is where **Dynamic Programming** enters. Dynamic Programming is one of the best methods for solving the 0/1 Knapsack Problem because it allows us to break the problem into smaller sub-problems, store previously computed results, avoid repeating unnecessary work, and gradually build the optimal solution

Not through guessing. Not through brute force. But through structured decision-making. And in this section we will finally see how.

1. Problem Definition

Let us consider the following 0/1 Knapsack Problem. Imagine you are given:

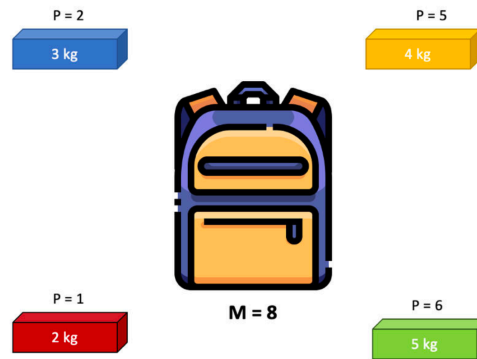
- A bag with limited capacity
- Several objects with different weights and profits

Your objective is simple: **Maximize the total profit, without exceeding the bag's capacity.**

But remember the important rule: This is a **0/1** Knapsack Problem. So each object must be fully included or completely excluded. There is no such thing as "half an object".

Suppose the bag capacity is: $M = 8$, and the number of objects is: $n = 4$

The profits and weights are: $P = [1, 2, 5, 6]$ and $W = [2, 3, 4, 5]$



We can summarize the problem like this:

Object	Profit	Weight
1	1	2
2	2	3
3	5	4
4	6	5

At first glance, this might look easy. But there is an immediate problem: We cannot simply place everything into the bag. The total weight would become: $2 + 3 + 4 + 5 = 14$. Which exceeds the capacity: $14 > 8$

So now we must make decisions. And every decision creates another possibility.

- Should we take Object 4?
- What if taking Objects 3 and 4 together is impossible?
- What if Objects 2 and 3 produce a better total profit?

This is where the complexity begins. In the naive approach, we try **every possible combination**. Each object has only two choices: included (1) or excluded (0)

So possible solutions look like:

```
{0, 0, 0, 0}
{1, 1, 1, 1}
{0, 0, 0, 1}
{0, 0, 1, 0}
{0, 1, 1, 0}
```

... and so on. Each position represents one object.

For example: `{0, 1, 1, 0}` means: Object 1 → not included; Object 2 → included; Object 3 → included; Object 4 → not included

Since each object has two possibilities, the total number of solutions becomes: 2^n

With: $n = 4$, we get: $2^4 = 16$ possible combinations. And that number grows extremely fast. If: $n = 20$, then: $2^{20} = 1,048,576$ possible solutions already exist.

This is why brute force becomes impractical very quickly. Its time complexity is: $O(2^n)$, which is exponentially expensive.

And this is exactly why Dynamic Programming matters. Instead of directly testing all possible solutions at once, Dynamic Programming solves the problem in a much smarter way. It breaks the large problem into smaller sub-problems, stores their answers, and gradually builds the optimal solution using **tabulation**.

So while brute force blindly explores everything, Dynamic Programming remembers what it has already learned. And that changes the problem completely.

2. Workflow

Now comes the interesting part. Earlier, brute force tried to explore every possible combination directly. And honestly, that becomes chaotic very quickly. Dynamic Programming approaches the problem differently.

- Instead of asking: “What is the final best combination?”, it asks something much smarter: “What is the best possible profit for every smaller capacity along the way?”

That single idea changes everything. To solve the 0/1 Knapsack Problem using Dynamic Programming, we create a table. The table contains: $(n + 1)$ rows and $(M + 1)$ columns

Since: $n = 4$ and $M = 8$, the table becomes:

- **5 rows** → from (0) to (4)
- **9 columns** → from (0) to (8)

The columns represent the maximum capacity of the bag.

But here's the important detail: We do **not** immediately solve for capacity 8. Instead, we gradually build the solution:

What if the bag capacity is 0? → What if it is 1? → What if it is 2? → 3? → 4? → ... → Until eventually reaching 8.

This is the essence of **bottom-up Dynamic Programming**. We solve many smaller problems first, then use them to solve larger ones.

Initially, the table is completely empty.

Objects (P, W)	0	1	2	3	4	5	6
0 Objects							
(1, 2)							
(2, 3)							
(5, 4)							
(6, 5)							

The values placed inside the table represent **the maximum obtainable profit at that specific capacity and object level**.

Step 1 — First Row

We begin with row index 0. This row means: *No objects are considered*. If there are no objects, then no profit can exist.

So the entire row becomes 0.

Objects (P, W)	0	1	2	3	4	5	6
0 Objects	0	0	0	0	0	0	0
(1, 2)							
(2, 3)							
(5, 4)							
(6, 5)							

Similarly: If the bag capacity is 0, then no object can be inserted either. So the first column will always remain 0.

The core idea during the entire process is this: “What happens if I start including this object?”. Every row represents a new level of decisions.

Step 2 — First Object

Now we move to row index 1. The first object has: **Profit = 1** and **Weight = 2**

This object can only fit when the capacity becomes at least 2. So:

- Capacities 0 and 1 → cannot include the object
- Capacities 2 to 8 → object can be included

The profit becomes 1.

Objects (P, W)	0	1	2	3	4	5	6
0 Objects	0	0	0	0	0	0	0
(1, 2)	0	0	1	1	1	1	1
(2, 3)							
(5, 4)							
(6, 5)							

Notice something important. When considering the current row, we only consider the current object and all previous objects. Future objects are ignored for now.

Step 3 — Second Object

Now move to row index 2. The second object has: **Profit = 2** and **Weight = 3**

This object can only fit when capacity reaches at least 3. At first:

- Capacity 3 → profit becomes 2
- Capacity 4 → profit remains 2

But now Dynamic Programming starts comparing combinations.

For example, at capacity 5: First object + second object; Weight = $(2 + 3 = 5)$; Profit = $(1 + 2 = 3)$

So capacity 5 onwards becomes profit 3.

Objects (P, W)	0	1	2	3	4	5	6
0 Objects	0	0	0	0	0	0	0
(1, 2)	0	0	1	1	1	1	1
(2, 3)	0	0	1	2	2	3	3
(5, 4)							
(6, 5)							

For instance, at capacity 4:

- Using Object 2 alone gives profit 2
- Using Object 1 alone gives profit 1
- Combining them exceeds capacity

So the best profit becomes 2. This is why Dynamic Programming always chooses the maximum possible profit at every step.

Step 4 — Third Object

Now we move to row index 3. The third object has: **Profit = 5** and **Weight = 4**

Immediately, something interesting happens. The profit 5 alone is already greater than all previous profits. So:

- Capacity 4 → profit becomes 5
- Capacity 5 → still 5

Now combinations begin again.

- Third object + first object: $4 + 2 = 6 \rightarrow$ Profit: $5 + 1 = 6$
- Third object + second object: $4 + 3 = 7 \rightarrow$ Profit: $5 + 2 = 7$
- All three objects together: $2 + 3 + 4 = 9$, which exceeds the capacity. So it is discarded.

Objects (P, W)	0	1	2	3	4	5	6
0 Objects	0	0	0	0	0	0	0

Objects (P, W)	0	1	2	3	4	5	6
(1, 2)	0	0	1	1	1	1	1
(2, 3)	0	0	1	2	2	3	3
(5, 4)	0	0	1	2	5	5	6
(6, 5)							

Notice again: Whenever no better profit is found, the value is copied from the previous row. Dynamic Programming never forgets previous optimal results.

Step 5 — Fourth Object

Finally, we reach row index 4. The fourth object has: **Profit = 6** and **Weight = 5**. The same process continues.

After comparing all valid combinations, we obtain the final table:

Objects (P, W)	0	1	2	3	4	5	6
0 Objects	0	0	0	0	0	0	0
(1, 2)	0	0	1	1	1	1	1
(2, 3)	0	0	1	2	2	3	3
(5, 4)	0	0	1	2	5	5	6
(6, 5)	0	0	1	2	5	6	6



Notice that **if a single object already cannot fit into the current bag capacity, then combining it with more objects will also never fit.**

The final answer becomes: 8, which is located at: $V[4, 8]$

The quick formula used during the process is:

$$V[i, W] = \max(V[i - 1, W], V[i - 1, W - W_i] + P_i)$$

This formula compares two choices:

1. Ignore the current object
2. Include the current object
3. And then selects the better profit.

For example, to calculate: $V[4, 1]$, we obtain: $V[4, 1] = \max(V[3, 1], V[3, -4] + 6)$

But: $V[3, -4]$ does not exist. So the current object cannot fit. Therefore: $V[4, 1] = 0$, which simply copies the value from the previous row.

Backtracking the Solution

Now comes the final part. We already know the maximum profit is: 8. But which objects created that profit? To find the answer, we backtrack through the table.

The idea is simple:

- If the value exists in both current and previous rows → object not included
- If the value changes → object included

Starting from: $V[4, 8] = 8$, we compare it with the previous row.

- Profit 8 does not exist in row 3. So Object 4 is included. Remaining profit: $8 - 6 = 2$
- Move upward. Profit 2 exists in both rows for Object 3. So Object 3 is not included.

- Move upward again. Profit 2 appears only after including Object 2. So Object 2 is included. Remaining profit: $2 - 2 = 0$
- Once the remaining profit becomes 0, no further objects are needed. So Object 1 is excluded.
- The final solution becomes: `{0,1,0,1}`. Meaning:
 - Object 1 → excluded
 - Object 2 → included
 - Object 3 → excluded
 - Object 4 → included

Time Complexity

The time complexity of Dynamic Programming for the 0/1 Knapsack Problem is: $O(N \times W)$, because the table contains: N rows and W columns

Unlike brute force: $O(2^n)$. Dynamic Programming avoids redundant calculations by storing intermediate results. And this is why it becomes dramatically faster for large problems.

Below is the simple Python implementation of 0/1 Knapsack Problem:

```
def knapsack_dp(values, weights, capacity):
    n = len(values)
    table = [[0]*(capacity + 1) for y in range(n + 1)]

    for i in range(1, n+1):
        for w in range(1, capacity+1):
            if weights[i-1] <= w:
                include_item = values[i-1] + table[i-1][w-weights[i-1]]
                exclude_item = table[i-1][w]
                table[i][w] = max(include_item, exclude_item)
            else:
                table[i][w] = table[i-1][w]

    for row in table:
        print(row)
    return table[n][capacity]
```

The function requires a list of values, a list of weights, and the maximum capacity of the bag. Ensure `list(values) = list(weights)`

```
values = [2, 5, 1, 6]
weights = [3, 4, 2, 5]
capacity = 8
max_value = knapsack_dp(values, weights, capacity)
print("Maximum value:", max_value)
```

```
[0, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 2, 2, 2, 2, 2]
[0, 0, 0, 2, 5, 5, 5, 7]
[0, 0, 1, 2, 5, 5, 6, 7]
[0, 0, 1, 2, 5, 6, 6, 8]
Maximum value: 8
```

Summary

The 0/1 Knapsack Problem demonstrates one of the most important strengths of Dynamic Programming:

▮ ***Solving large problems by remembering smaller solutions.***

Unlike the Greedy approach, which only focuses on immediate choices, Dynamic Programming systematically explores combinations while avoiding unnecessary repeated calculations.

Instead of testing every possible solution directly, it gradually builds answers using a table of previously computed profits. This transforms an exponential-time problem into a much more manageable one.

Throughout the process, we learned that every row represents another level of object consideration, every column represents another possible capacity, and every cell stores the best obtainable profit at that state

And by comparing: **including the object versus excluding it**, the algorithm continuously constructs the optimal solution step by step. Finally, by tracing backward through the completed table, we can determine exactly which objects generated the maximum profit.

So in the end, the 0/1 Knapsack Problem is not simply about filling a bag. It is about structured decision-making. Dynamic Programming teaches the machine to remember the past, so it can make better decisions for the future.

References

<https://www.geeksforgeeks.org/dsa/0-1-knapsack-problem-dp-10/>

<https://www.youtube.com/watch?v=nLmhmb6NzcM&themeRefresh=1>

<https://www.youtube.com/watch?v=qxWu-SeAqe4>